**Doc. no.:** P0602R3
**Date:** 2018-06-13
**Audience:** Library Working Group
**Reply-to:** Zhihao Yuan <zy at miator dot net>

# variant and optional should propagate copy/move triviality

## Rationale

Making `variant` and `optional` conditionally trivially copy constructible, trivially move constructible, trivially copy assignable, and trivially move assignable significantly improves codegen and performance when they are nested or being put into containers.

Given the spirit of the resolution of LWG 2900 and feedback from implementers, this paper proposes a full-fledged approach shipped in libc++, MS STL, and libstdc++ that is to make the four operations individually trivial if all alternatives have the corresponding operations being trivial.

This change comes with a (good) potential ABI breakage in user's code – a small enough trivially copyable `optional` & `variant` took or returned by a function can be passed via register after applying the change, therefore we propose to accept this change as a defect report against C++17.

## Wording

This wording is relative to N4750.

Modify 23.6.3.1 [optional.ctor] as follows:

```
constexpr optional(const optional& rhs);
```

[...]

*Remarks:* This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` is `true`. If `is_trivially_copy_constructible_v<T>` is `true`, this constructor ~~shall be a `constexpr` constructor~~is trivial.

```
constexpr optional(optional&& rhs) noexcept(see below );
```

[...]

*Remarks:* The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<T>`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T>` is `true`. If `is_trivially_move_constructible_v<T>` is `true`, this constructor ~~shall be a `constexpr` constructor~~is trivial.

Modify 23.6.3.3 [optional.assign] as follows:

```
optional<T>& operator=(const optional& rhs);
```

[...]

*Remarks:* If any exception is thrown, [...]. This operator shall be defined as deleted unless `is_copy_constructible_v<T>` is `true` and `is_copy_assignable_v<T>` is `true`. If `is_trivially_copy_constructible_v<T> && is_trivially_copy_assignable_v<T> && is_trivially_destructible_v<T>` is `true`, this assignment operator is trivial.

```
optional<T>& operator=(optional&& rhs) noexcept(see below );
```

[...]

*Remarks:* The expression inside `noexcept` is equivalent to: [...] This operator shall not participate in overload resolution unless `is_move_constructible_v<T>` is `true` and `is_move_assignable_v<T>` is `true`. If `is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T> && is_trivially_destructible_v<T>` is `true`, this assignment operator is trivial.

Modify 23.7.3.1 [variant.ctor] as follows:

```
variant(const variant& w);
```

[...]

*Remarks:* This constructor shall be defined as deleted unless `is_copy_constructible_v<`$T_i$`>` is `true` for all *i*. If

`is_trivially_copy_constructible_v<T_i>` is `true` for all *i*, this constructor is trivial.

```
variant(variant&& w) noexcept(see below );
```

[...]

*Remarks:* The expression inside `noexcept` is equivalent to [...]. This function shall not participate in overload resolution unless `is_move_constructible_v<T_i>` is `true` for all *i*. If `is_trivially_move_constructible_v<T_i>` is `true` for all *i*, this constructor is trivial.

Modify 23.7.3.3 [variant.assign] as follows:

```
variant& operator=(const variant& rhs);
```

[...]

*Remarks:* This operator shall be defined as deleted unless `is_copy_constructible_v<T_i> && is_copy_assignable_v<T_i>` is `true` for all *i*. If `is_trivially_copy_constructible_v<T_i> && is_trivially_copy_assignable_v<T_i> && is_trivially_destructible_v<T_i>` is `true` for all *i*, this assignment operator is trivial. [...]

```
variant& operator=(variant&& rhs) noexcept(see below );
```

[...]

*Remarks:* This function shall not participate in overload resolution unless `is_move_constructible_v<T_i> && is_move_assignable_v<T_i>` is `true` for all *i*. If `is_trivially_move_constructible_v<T_i> && is_trivially_move_assignable_v<T_i> && is_trivially_destructible_v<T_i>` is `true` for all *i*, this assignment operator is trivial. The expression inside `noexcept` is equivalent to: [...]

## Acknowledgments

Thanks Casey Carter for evolving the implementations and explaining the results.